

PORTFOLIO

끊임없이 성장하려고 노력하며 타인의 말을 항상 귀기울이는 개발자

권민석

chris0540@naver.com

010 - 9758 - 0540

Kwon Minsuk

백엔드 엔지니어 | 권민석



Birth. [2000.03.23](#)
E-mail. chris0540@naver.com
Tel. [010.9758.0540](tel:010.9758.0540)
GitHub. github.com/RIMIN0650

학력사항

2020.03~2026.02 가천대학교 졸업 (3.85)

자격사항

정보처리기사 2025.02
SQLD 2025.06

활동

2023.12 ~ 2024.05 [메가스터디 IT \(백엔드\)](#)
2025.02 ~ 2025.02 [제 3회 와글와글 해커톤 \(백엔드\)](#)
2025.12 ~ 2026.06 [한화시스템 BEYOND 캠프 \(백엔드\)](#)
2026.08 ~ [서비스 성능 최적화 및 인프라 구축 역량 강화](#)

보유스킬

Backend [Java](#), [Spring Boot](#), [Spring Data JPA](#)
Spring Security, Spring Batch
Database [MySQL](#), [MariaDB](#)
Frontend Vue.js
DevOps Docker, Kubernetes, AWS EC2, Linux
Testing nGrinder, VisualVM

CONTENTS

Nexus

01

가맹점 POS 결제 및 AI 자동 발주부터
본사의 재고·물류·정산·통계 분석까지
전 과정을 통합 관리하는
MSA 기반 프랜차이즈 플랫폼

- Spring Security
- Spring Batch

MediQ

02

위치 기반 서비스 기술을 활용한
실시간 병원 탐색 및 원격 진료 대기 예약 플랫폼

- WebSocket
- Web Push

TECH 1: SPRING BATCH

정기결제 자동화와 대용량 정산 처리를 위한 배치 시스템 구현

NEXUS 

기간 : 26.04-26.06

- JAVA
- Spring Boot
- Spring Batch
- JPA
- MariaDB



정기결제 자동화

Billing Key 기반 정기 결제 자동화

대용량 정산 처리

Spring Batch + Chunk 기반 대용량 처리

N + 1 문제 해결

BATCH 내 N + 1 제거 및 DB 집계 최적화

결제 실패 재처리

실패 건 재처리 + 실패 사유 관리

SPRING BATCH : 사용 이유

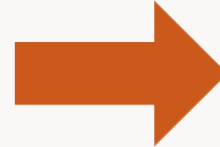
문제점



@Scheduled 방식의 한계

서비스 규모 증가로 대량의 정산 데이터를 처리하면서 메모리 사용량이 증가하고, 작업 실패 시 실패지점부터 작업을 재개하기 어려운 문제 발생

- 대량의 가맹점의 주문 데이터를 한 번에 처리하면서 메모리 사용량 및 처리 시간 증가
- 작업 중 장애 발생 시 실패 지점 관리가 어려워 전체 작업 재실행에 대한 부담 발생



해결



Spring Batch 도입

자동 정기 결제 서비스를 MSA로 분리하고 Spring Batch를 도입하여, 대용량 데이터를 안정적으로 처리할 수 있는 배치 아키텍처를 구현

- Chunk + Paging
대량 데이터를 페이지 단위로 조회하고 Chunk 단위로 처리하여 메모리 사용량을 제한
- JOB / Step 단위로 작업을 분리하고 실행 상태를 관리하여 작업 실패 시 실패한 작업을 재시작할 수 있도록 개선

DB 집계 최적화 결과 | JVM HEAP 사용량 개선

기존 방식 | 전체 ENTITY 전체 조회 후 JVM 적재

```
for (Orders orders : lastMonthOrders) {  
    totalPayAmount += orders.getPrice();  
}
```

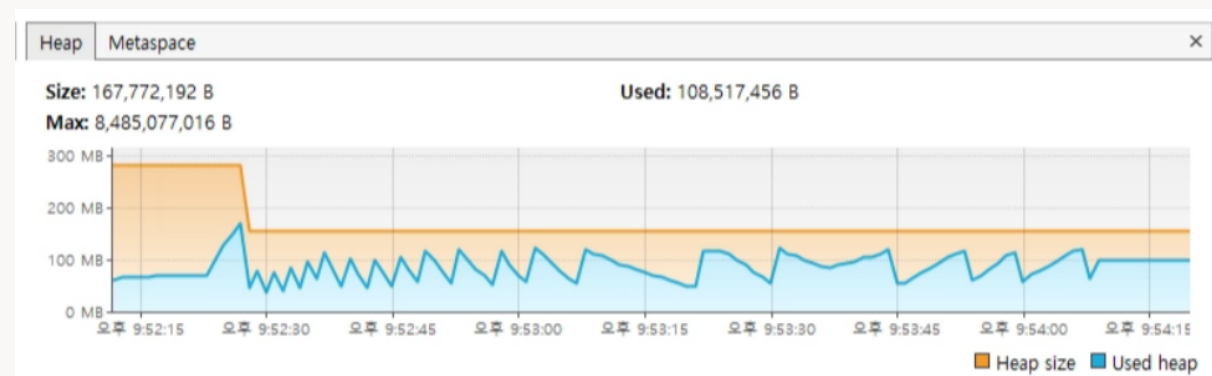
Orders Entity 전체를 JVM에 적재한 후
Java 반복문으로 주문 금액을 합산

개선 방식 | DB SUM 집계

```
@Query(""  
    SELECT COALESCE(SUM(o.price), 0)  
    FROM Orders o "  
    "WHERE o.store.idx = :storeId "  
    "AND o.createdAt >= :startDate "  
    "AND o.createdAt < :endDate")
```

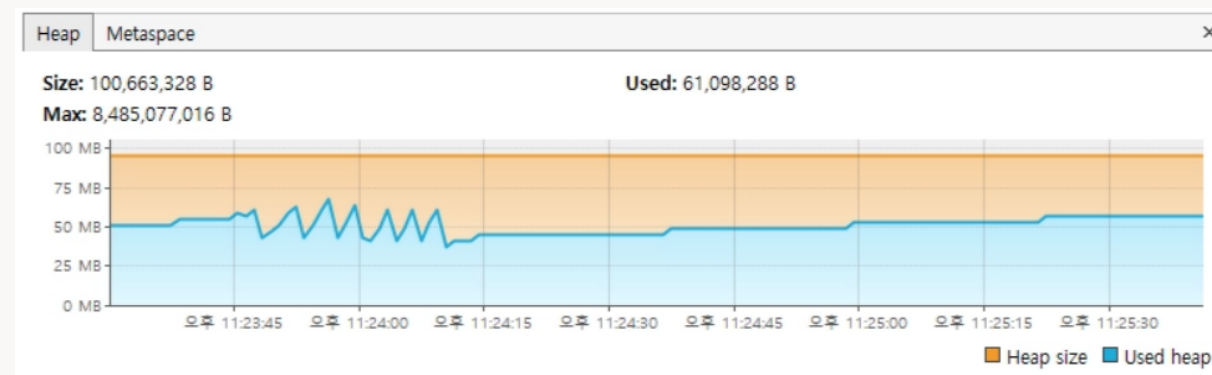
DB에서 주문 금액을 직접 집계하여
합산 결과만 JVM으로 전달

DB 집계 최적화 결과 : JVM HEAP



개선 전

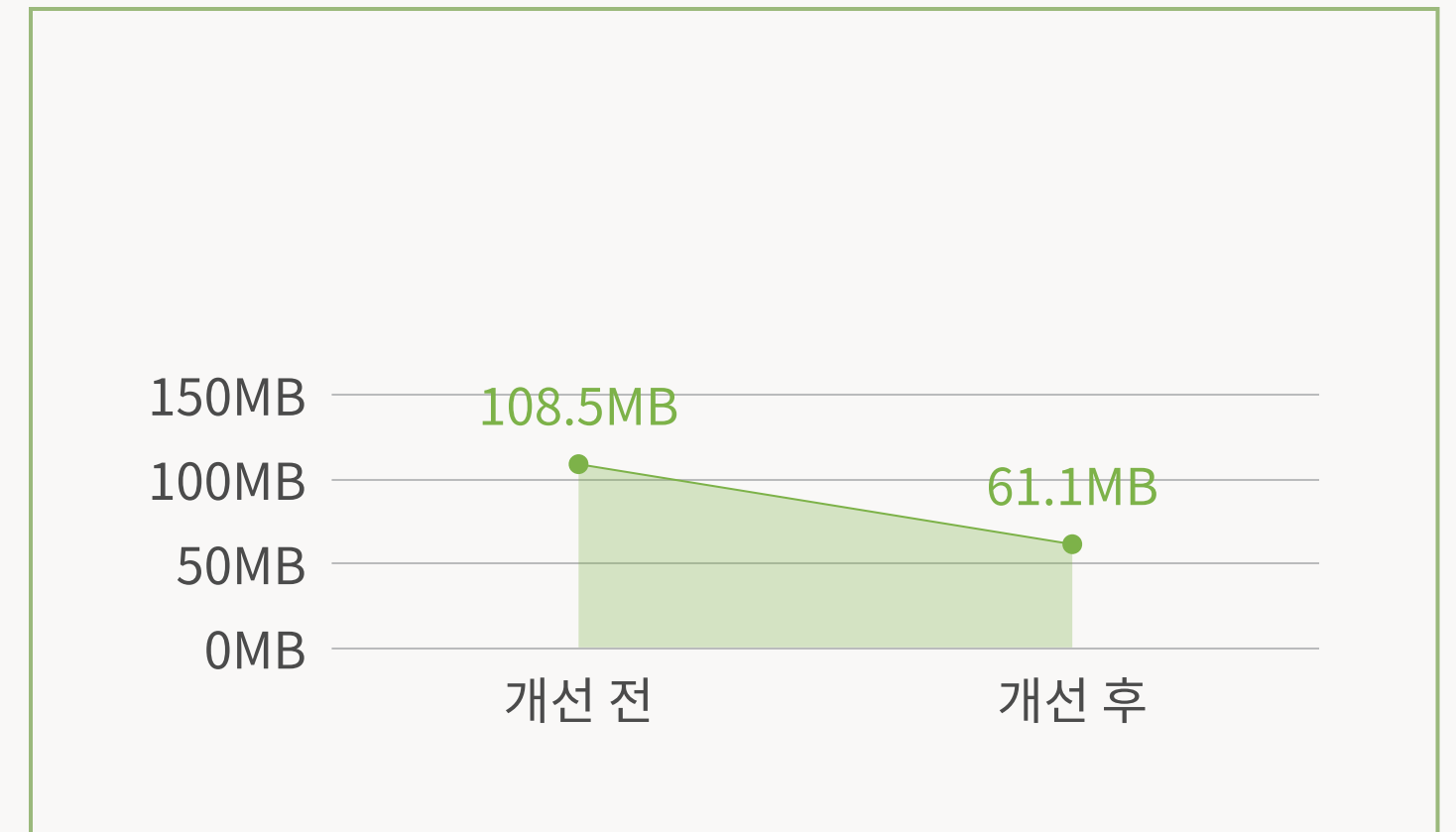
- 주문 데이터를 JVM으로 전체 조회
- JAVA 반복문으로 금액 합산
- 다수의 ORDERS ENTITY가 HEAP에 적재
- 객체 생성 및 GC 반복 발생



개선 후

- 주문 데이터를 JVM 으로 로딩하지 않고 DB 에서 SUM() 처리하도록 변경
- JVM에 적재되는 ORDERS ENTITY 감소
- 불필요한 객체 생성 및 GC 부담 감소
- 집계 결과만 반환하여 메모리 사용량 최적화

VISUALVM을 활용한 JVM HEAP USAGE 측정



USED HEAP 108.5MB → 61.1MB
약 43.7 % 감소

성능 개선 : N + 1 제거

기존 방식 | 가맹점별 개별 집계

```
for (Store store : stores) {  
    int totalPayAmount =  
        ordersRepository.sumPriceByStoreAndPeriod(  
            store.getIdx(),  
            startDate,  
            endDate  
        );  
}
```

가맹점마다 주문 금액 집계 Query를 실행

Store N 개 = Query N회

개선 방식 | GROUP BY 기반 일괄 조회

```
List<Object[]> results =  
    ordersRepository.sumPriceByStoreIdxsAndPeriod(  
        storeIdxs, startDate, endDate);
```

```
SELECT store_id, SUM(price)  
FROM orders  
WHERE store_id IN (...)  
GROUP BY store_id
```

1. Reader에서 Store ID 목록 수집
2. IN + GROUP BY + SUM 으로 일괄 집계
3. 가맹점 별 결과를 일괄 반환
4. Processor에서 추가 DB 조회 없이 결과 활용

N + 1 제거 결과 : 쿼리 및 처리 시간 개선

N + 1로 인해 반복 발생하던 주문 집계 쿼리를 일괄 집계 방식으로 변경

 batch  2026-08-24 오후 11:24

 [정산 배치] 작업 시작 - 일시: 2026-08-24 23:23:45

 [정산 배치] 작업 종료

- 상태: COMPLETED
- 시작 시간: 2026-08-24 23:23:45
- 종료 시간: 2026-08-24 23:24:10
- 총 건수: 1000
- 성공: 1000
- 실패: 0

"배치 작업 중 실행된 쿼리 수: 3402"

개선 전

실행된 쿼리 수 : 3402

소요 시간 : 25초

 batch  2026-09-01 오전 11:05

 [정산 배치] 작업 시작 - 일시: 2026-09-01 11:04:41

 [정산 배치] 작업 종료

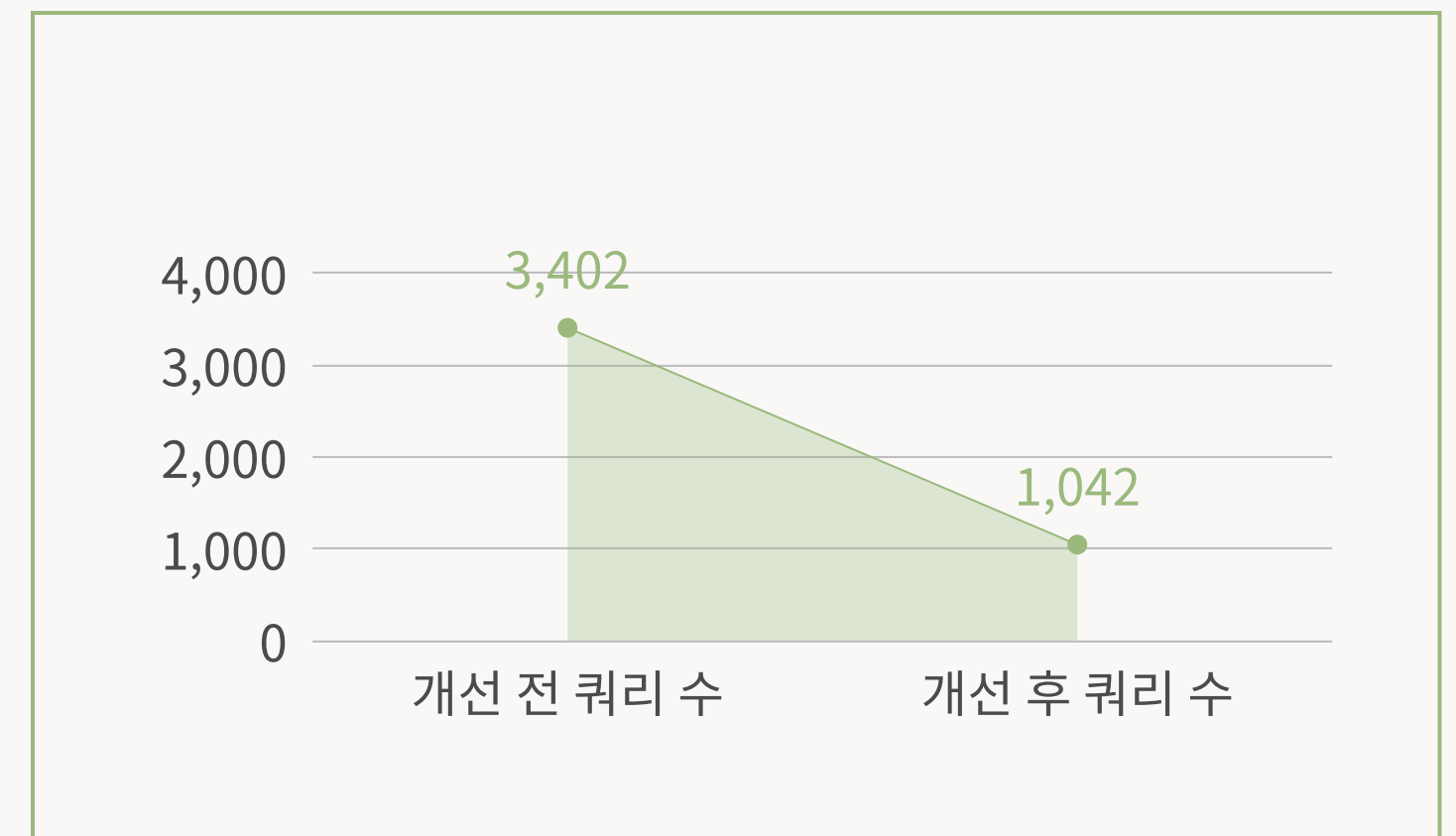
- 상태: COMPLETED
- 시작 시간: 2026-09-01 11:04:41
- 종료 시간: 2026-09-01 11:04:49
- 총 건수: 1000
- 성공: 1000
- 실패: 0

"배치 작업 중 실행된 쿼리 수: 1042"

개선 후

실행된 쿼리 수 : 1042

소요 시간 : 8초



테스트 결과 쿼리 수 : 3402 → 1042 (약 69.4% 감소)
소요시간 : 25초 → 8초 (약 68% 감소)

TECH 2: SPRING SECURITY

JWT 기반 Stateless 인증 구조를 설계하고
Spring Security 인증 흐름을 커스터마이징



01 JWT 기반 STATELESS 인증

세션 기반 인증 대신 JWT를 도입하여 서버가 인증 상태를 저장하지 않는 Stateless 인증 구조를 구현

- JWT 기반 ACCESS TOKEN 인증
- SESSION 미사용

02 CUSTOM AUTHENTICATION FILTER 구현

UsernamePasswordAuthenticationFilter를 확장하여 JSON 기반 로그인 요청을 처리하고
인증 성공 시 JWT를 발급하도록 구성

- JSON LOGIN 요청 처리
- JWT 발급 및 인증 흐름 커스터마이징

03 SECURITY CONTEXT 기반 인증 연동

JWT에서 사용자 정보를 추출하여 Authentication 객체를 생성하고
SecurityContext에 등록하여 인증된 사용자로 인식하도록 구현

- AUTHENTICATION 객체 생성
- SECURITYCONTEXTHOLDER 인증 정보 등록

04 SECURITY FILTERCHAIN 구성

SecurityFilterChain을 커스터마이징하여 JWT 인증 필터의 실행 순서와 인증·인가 정책을 구성

- JWT FILTER 실행 순서 제어
- 인증 인가 정책 구성

SPRING SECURITY : WHY JWT?

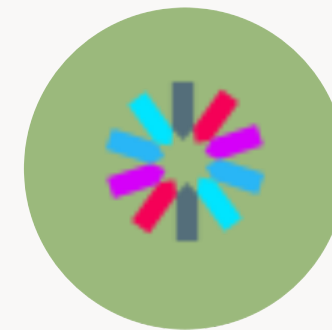
상황



세션 기반 인증의 서버 상태 의존성

- 인증 상태를 서버 세션에서 관리
- 서버 확장 시 세션 관리 고려 필요

해결



JWT 기반 Stateless 인증

- JWT에 인증 정보를 담아 서버 세션 의존성 제거
- 요청마다 JWT를 검증하여 인증 처리

JWT 인증 처리 흐름

로그인 시 JWT를 발급하고 요청마다 토큰을 검증

로그인 - JWT 발급

```
String Token = jwtUtil.createToken(...);  
response.setHeader(  
    "Set-Cookie",  
    "ACCESS_TOKEN=" + token + "; Path =/"  
);
```

로그인 성공 후 JWT를 생성하여 Cookie에 저장

API 요청 - JWT 인증

1. API 요청
2. CustomAuthenticationFilter
3. JWT 추출
4. JWT 검증
5. 사용자 정보 추출

요청마다 JWT를 검증하여 사용자 식별 정보와 권한 추출

JWT 인증과 Spring Security 연동

JWT 인증 정보를 Authentication과 SecurityContext로 연결

SecurityContext 인증 정보 등록

JWt에서 추출한 사용자 정보로 Authentication을 생성하고
SecurityContext에 등록



```
Authentication authentication =  
new UsernamePasswordAuthenticationToken(  
principal,  
null,  
authorities  
);  
SecurityContextHolder.getContext().  
setAuthentication(authentication);
```

CustomAuthenticaitonFilter를 통한 JWT 인증

UsernamePasswordAuthenticationFilter보다
먼저 JWT를 검증하도록 CustomAuthenticationFilter를 등록

```
http.addFilterBefore(  
    jwtFilter,  
    UsernamePasswordAuthenticationFilter.class  
);
```



JWT 인증 정보를 Spring Security 인증 체계와 연동

TECH 3: WEB PUSH

WEB PUSH 기반 실시간 알림 시스템 구현

MEDIQ



기간 : 26.04-26.06

위치 기반 서비스 기술을 활용한
실시간 병원 탐색 및 원격 진료 대기 예약 플랫폼



- PUSH API
- VAPID
- SERVICE WORKER

WEB PUSH : 브라우저 비활성 상태에서도 알림 전달

문제 상황



기존 알림 방식의 한계

웹 페이지가 활성화되지 않은 상태에서는
기존 페이지 기반 알림만으로 사용자에게
즉시 알림을 전달하기 어려움

해결



WEB PUSH 기반 알림 구조

WEB PUSH + SERVICE WORKER 기반 알림 구조 구현
브라우저의 PUSH SUBSCRIPTION을 서버에 등록하고
서버에서 PUSH SERVICE를 통해 알림 전달

WEB PUSH | 서버 PUSH 전송

```
NotificationEntity entity =  
    notificationRepository.findById(dto.getUserIdx())  
        .orElseThrow();  
Subscription subscription = new Subscription(  
    entity.getEndpoint(),  
    new Subscription.Keys(  
        entity.getP256dh(),  
        entity.getAuth()  
    )  
);  
Notification notification =  
    new Notification(  
        subscription,  
        NotificationDto.Payload.from(dto).toString()  
    );  
pushService.send(notification);
```

서버 PUSH 전송

저장된 사용자별 Push Subscription을 조회하여
Notification을 생성하고 Push Service로 전송

DB → Subscription → Notification → Push Service

WEB PUSH | Service Worker에서 알림 처리

```
self.addEventListener("push", (event) => {  
  const data = event.data.json();  
  event.waitUntil(  
    self.registration.showNotification(  
      data.title,  
      {body: data.message}  
    )  
  );  
});
```

SERVICE WORKER

페이지가 활성화되지 않은 상태에서도
PUSH EVENT를 수신하여 NOTIFICATION 표시

Push Event 수신 → Service Worker → Payload 파싱 → Notification 표시

TECH 4: Websocket



MEDIQ



기간 : 26.04-26.06

위치 기반 서비스 기술을 활용한
실시간 병원 탐색 및 원격 진료 대기 예약 플랫폼

WEB
SOCKET

실시간 대기열 상태 동기화

BROADCAST

변경된 대기열 상태 전파

WEB SOCKET : WHY?

문제 상황



POLLING

POLLING 방식은 일정 주기로 반복 요청하여 변경 여부를 확인

- 불필요한 반복 요청
- 변경 발생과 요청 시점 사이의 지연

해결

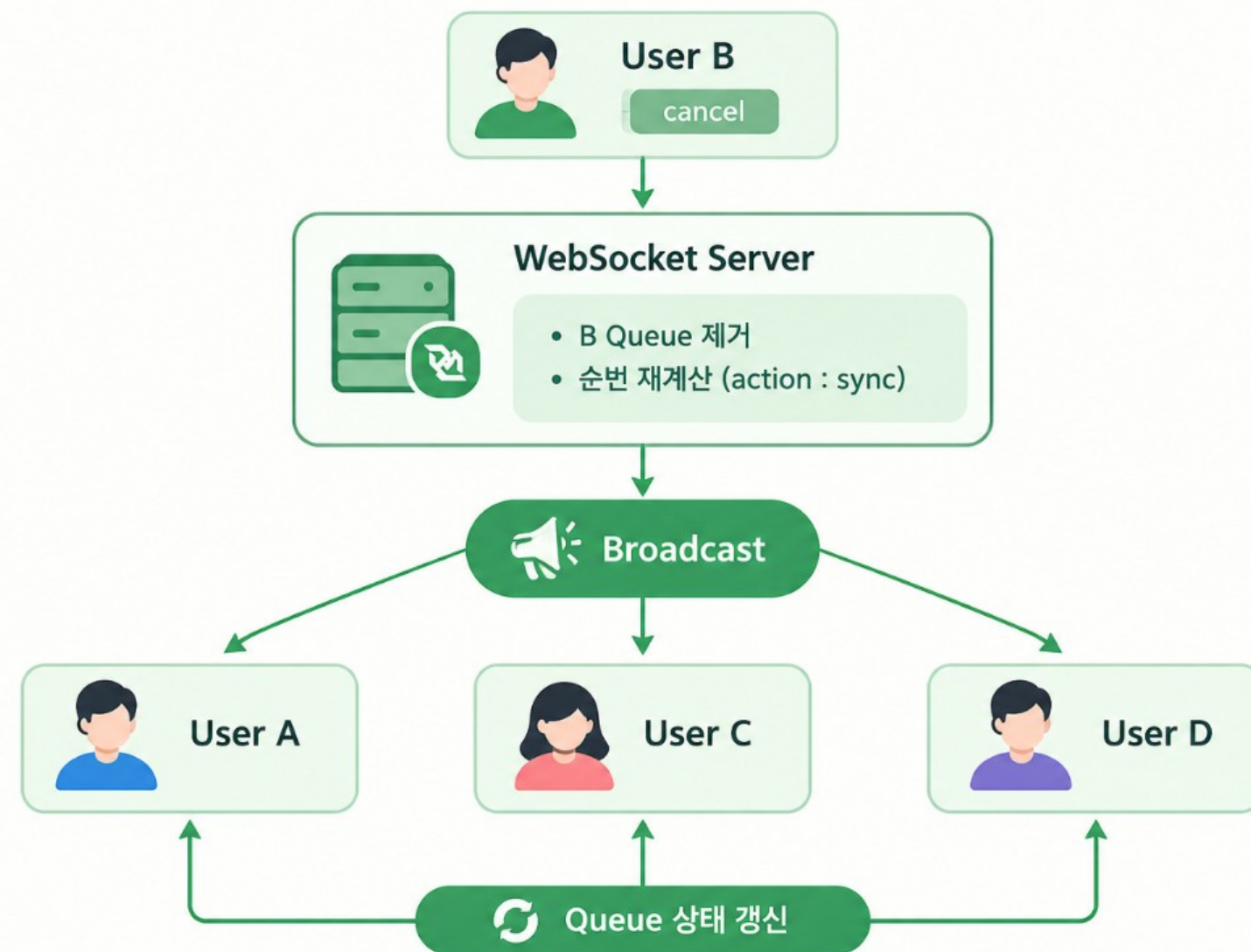


WEBSOCKET

WebSocket 연결을 유지하고 대기열 변경 발생 시 서버에서 즉시 상태 전파

- 연결을 유지하며 양방향 통신
- 서버의 Queue 변경 즉시 전달
- 연결된 사용자에게 변경 상태 동기화

WEBSOCKET | 대기열 변경 이벤트 전파



서버와 클라이언트 상태 동기화

대기열 변경 발생 시 서버에서 최신 상태를 계산하고 연결된 사용자에게 Broadcast

WEBSOCKET | 연결 및 실시간 메시지 처리



```
websocket = new WebSocket(wsUri)
  websocket.onopen = () => {
    connected.value = true
  }

websocket.onclose = () => {
  connected.value = false
}

websocket.onmessage = (e) => {
  const payload = JSON.parse(e.data)
  const data = payload.payload
    ? JSON.parse(payload.payload) : payload
  handleQueueUpdate(data)
}

function removeFromQueue(nickname) {
  ...
  updatePositions()
}
```

WEBSOCKET 연결 및 이벤트 처리

WEBSOCKET 연결 상태를 관리하고 서버에서 전달되는 대기열 변경 이벤트를 수신하여 클라이언트 상태를 갱신

- 연결 상태 관리
- 서버 이벤트 수신
- QUEUE 변경 이벤트 처리
- 순번 재계산
- 클라이언트 상태 갱신

THANK YOU

권민석

Backend Engineer

문제를 발견하고, 원인을 분석하고,
수치로 검증하며 개선하는 백엔드 엔지니어

GitHub	github.com/RIMIN0650
Email	chris0540@naver.com
Phone	010-9758-0540
Blog	https://song-gongming.tistory.com/